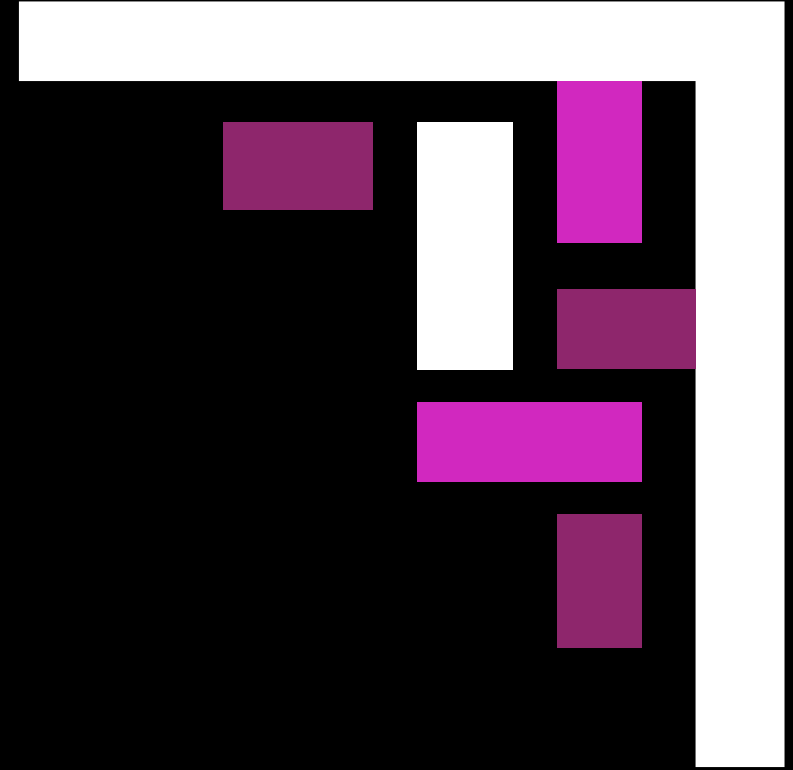
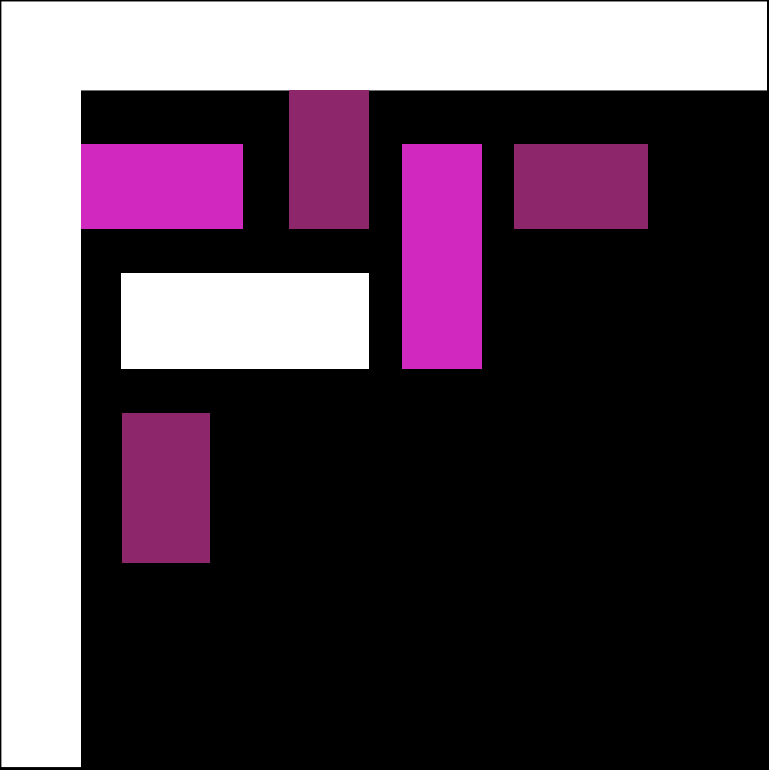
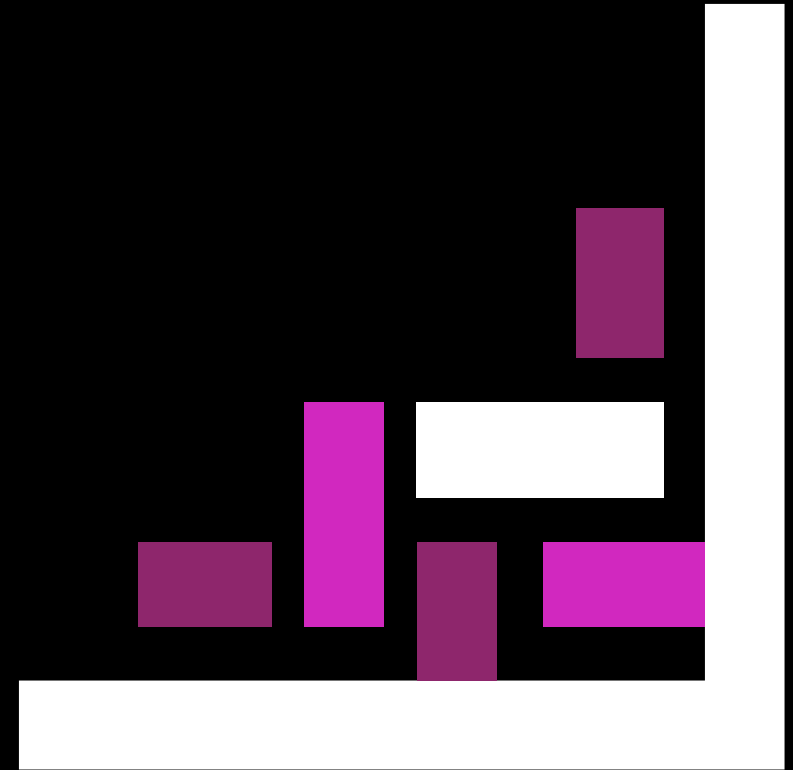
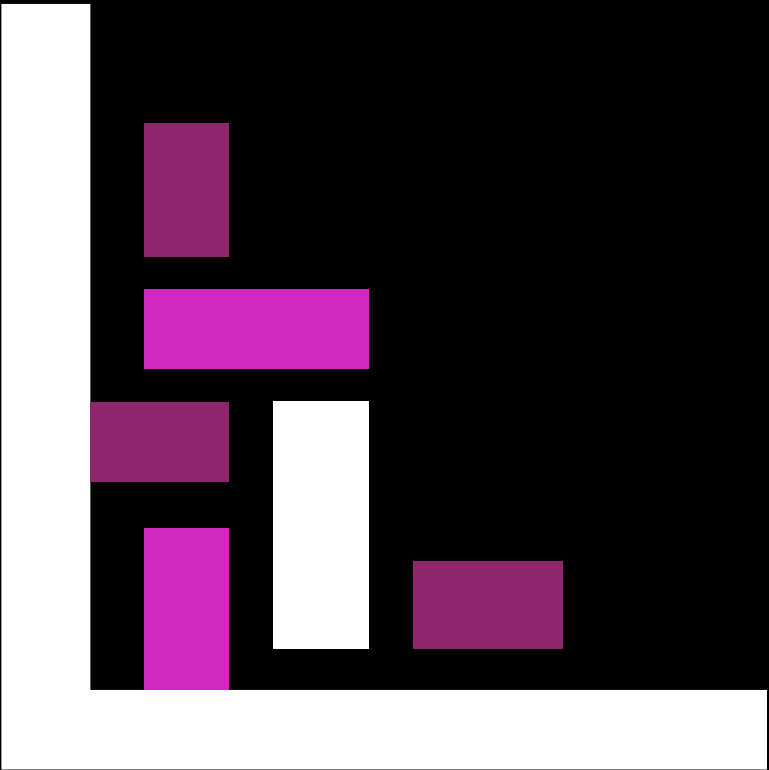


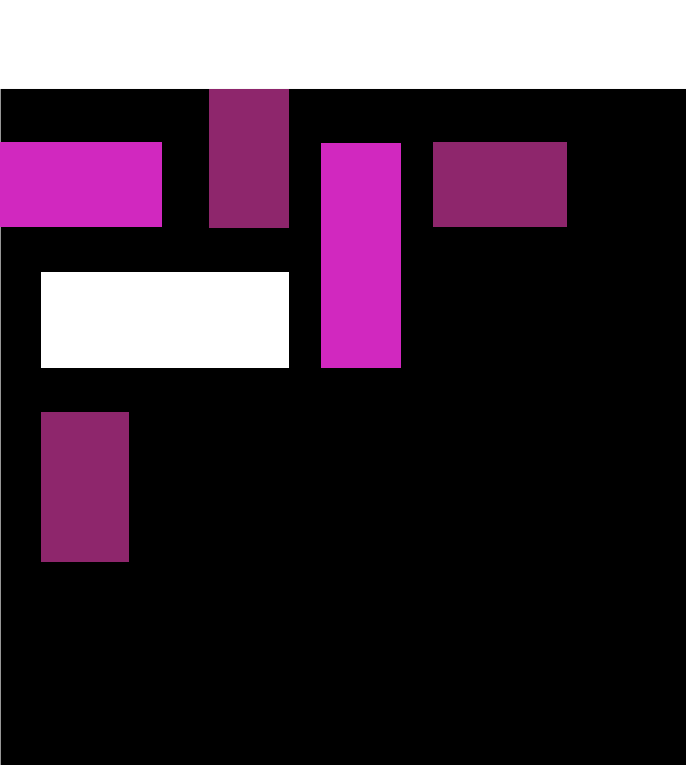


Presentación Técnica y Laboratorio Práctico

Tarea #3



26/07/2026



Integrantes Grupo #1

120140083 - Eduar Moreno

122070020 - Jose Sanchez

120160026 - Erasmo Mejía

122070150 - Bryan Campigottl

122079169 - Marco Izaguirre

121390061 - Oscar Montejo

222020008 - Fernando García

322450013 - Edar Murcia



Tarea 3

Introducción

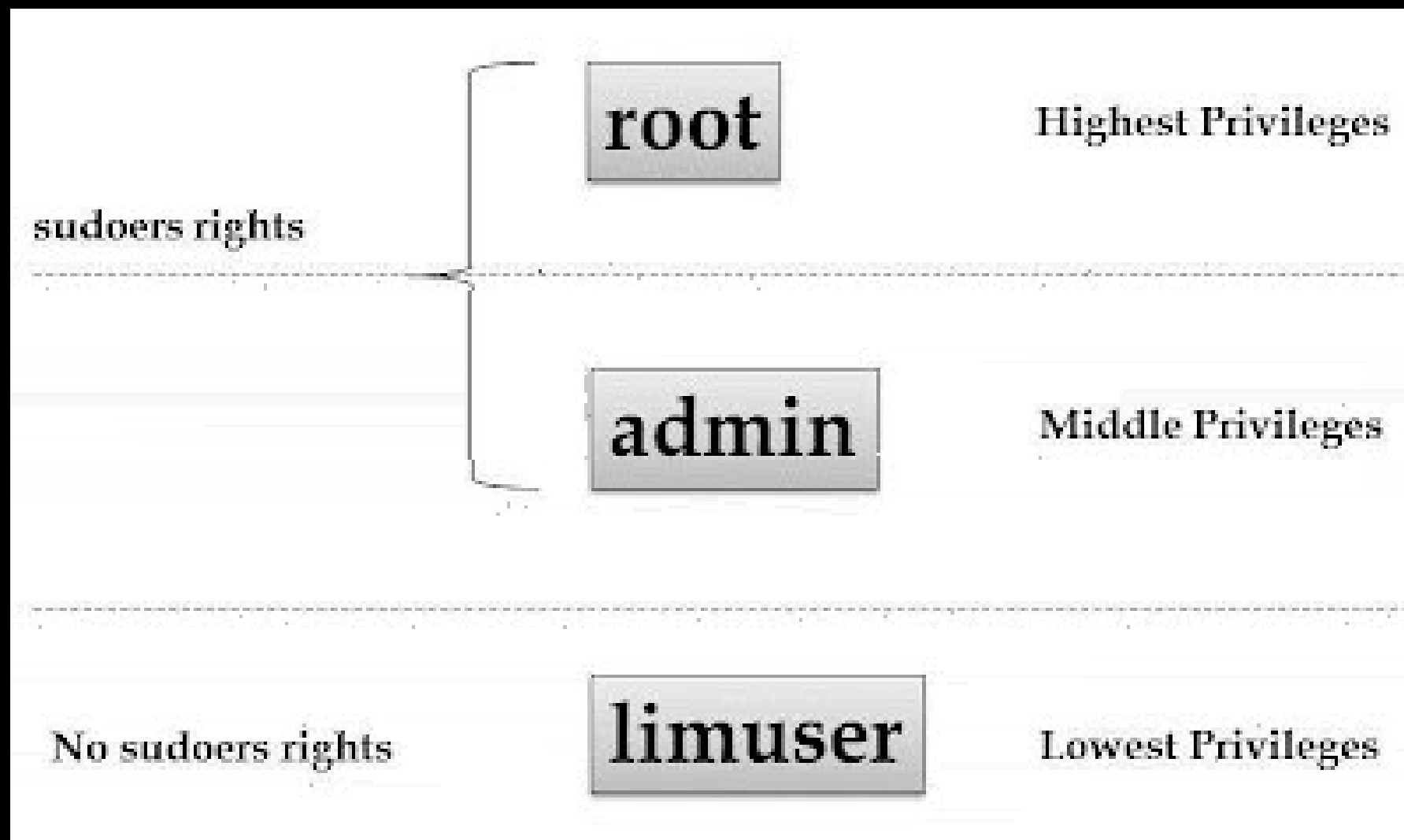
En este trabajo de grupo desarrollamos una aplicación sencilla en Node.js y la ejecutamos dentro de contenedores Docker. Nuestro objetivo fue comparar dos configuraciones: una versión insegura que corre como usuario root y otra versión segura que corre como usuario limitado. A lo largo de la práctica trabajamos en equipo para organizar los archivos, construir las imágenes y finalmente levantar los contenedores en el puerto 3000, mostrando en el navegador cómo ambas versiones funcionan y resaltando la importancia de aplicar buenas prácticas de seguridad.

Seguridad en Docker

La seguridad en Docker es un conjunto de prácticas y mecanismos diseñados para proteger los contenedores, las imágenes y la infraestructura donde se ejecutan. Aunque Docker proporciona aislamiento entre aplicaciones mediante contenedores, este aislamiento no es tan completo como el de una máquina virtual, ya que todos los contenedores comparten el mismo núcleo del sistema operativo.



Root vs Non-root User



En Docker, uno de los principios de seguridad más importantes es decidir si un contenedor debe ejecutarse como usuario root o como un usuario sin privilegios.

Esta decisión influye directamente en el nivel de acceso que tendrá la aplicación dentro del contenedor y en el impacto que podría tener un ataque si el contenedor llegara a ser comprometido.

USER en Dockerfile

Utilizar USER para ejecutar la aplicación con un usuario non-root es una de las mejores prácticas recomendadas por Docker. Al limitar los privilegios del proceso, se reduce el riesgo de que un atacante pueda modificar archivos del sistema, acceder a recursos sensibles o comprometer el sistema anfitrión en caso de que la aplicación presente una vulnerabilidad.

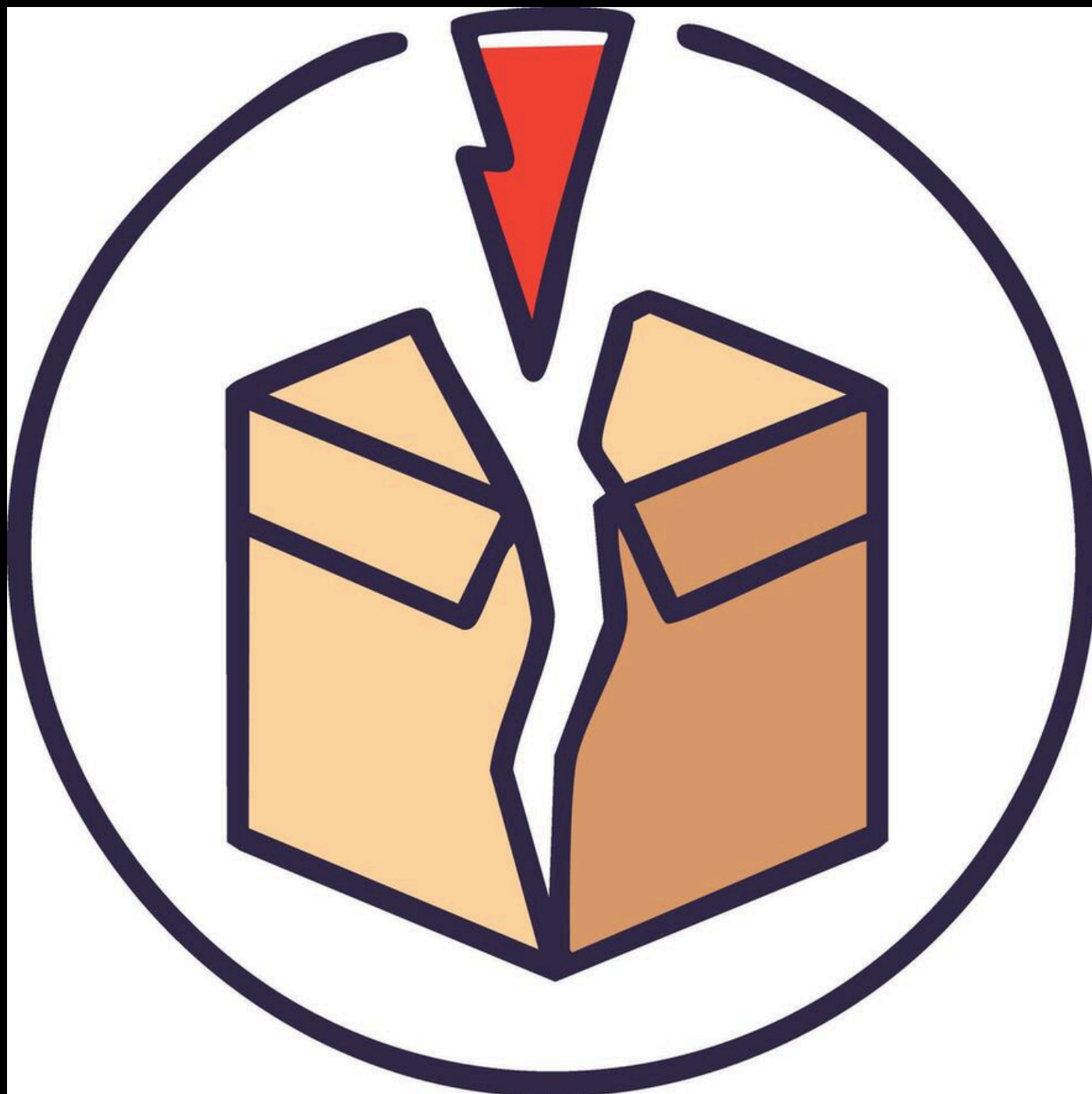


Docker Secrets

Docker Secrets es una función de Docker diseñada para almacenar y administrar información confidencial, como contraseñas, claves API, certificados y credenciales de bases de datos, de forma segura. Su objetivo es evitar que estos datos sensibles se incluyan directamente en las imágenes o en el código fuente de la aplicación.



Variables de entorno sensibles



Las variables de entorno sensibles son aquellas que contienen información confidencial, como contraseñas, claves API, tokens de acceso o credenciales de bases de datos, y que utilizan las aplicaciones para su configuración. Aunque las variables de entorno son una forma práctica de almacenar configuraciones, no son el método más seguro para manejar información crítica.

Imágenes oficiales

Las imágenes oficiales son imágenes de Docker mantenidas y verificadas por los desarrolladores del proyecto o por Docker Hub. Estas imágenes suelen ser más confiables, ya que reciben actualizaciones periódicas, correcciones de errores y parches de seguridad para reducir vulnerabilidades.



Docker Scout

Docker Scout es una herramienta de seguridad desarrollada por Docker que permite analizar imágenes de contenedores para identificar vulnerabilidades conocidas y evaluar su estado de seguridad. Su objetivo es ayudar a los desarrolladores a detectar riesgos antes de desplegar sus aplicaciones en producción.



Escaneo de vulnerabilidades



El escaneo de vulnerabilidades es el proceso de analizar imágenes y contenedores para identificar fallos de seguridad, bibliotecas desactualizadas o software con vulnerabilidades conocidas. Su principal objetivo es detectar estos riesgos antes de que las aplicaciones sean implementadas en producción.

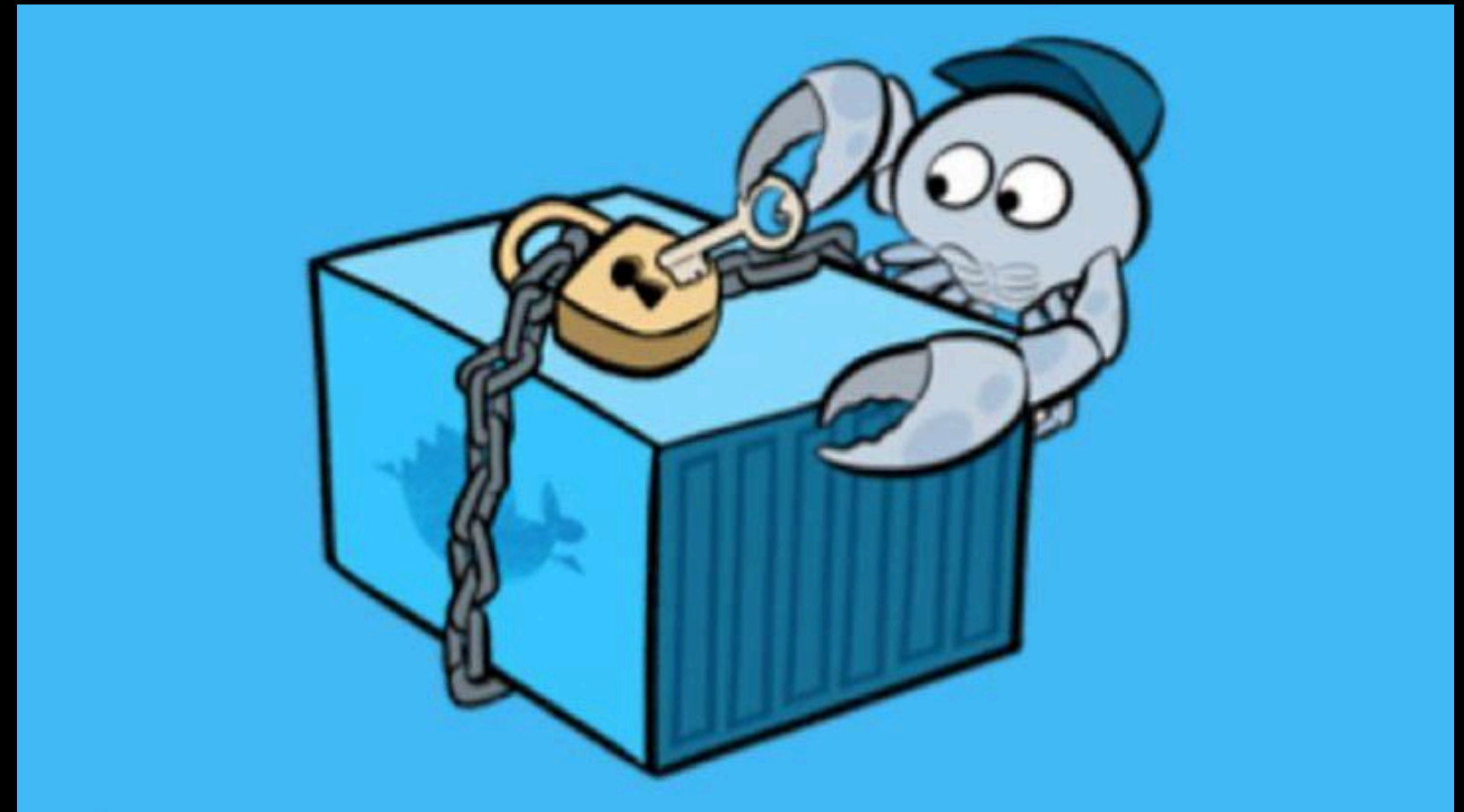
Capabilities

Las Capabilities en Docker son un mecanismo de seguridad de Linux que divide los privilegios del usuario root en permisos más específicos. En lugar de otorgar acceso total al contenedor, es posible conceder únicamente las capacidades que una aplicación necesita para funcionar.



Docker Content Trust

Docker Content Trust es una función de seguridad que permite verificar la autenticidad e integridad de las imágenes de Docker mediante firmas digitales. Su propósito es garantizar que una imagen provenga de una fuente confiable y que no haya sido modificada desde que fue publicada.

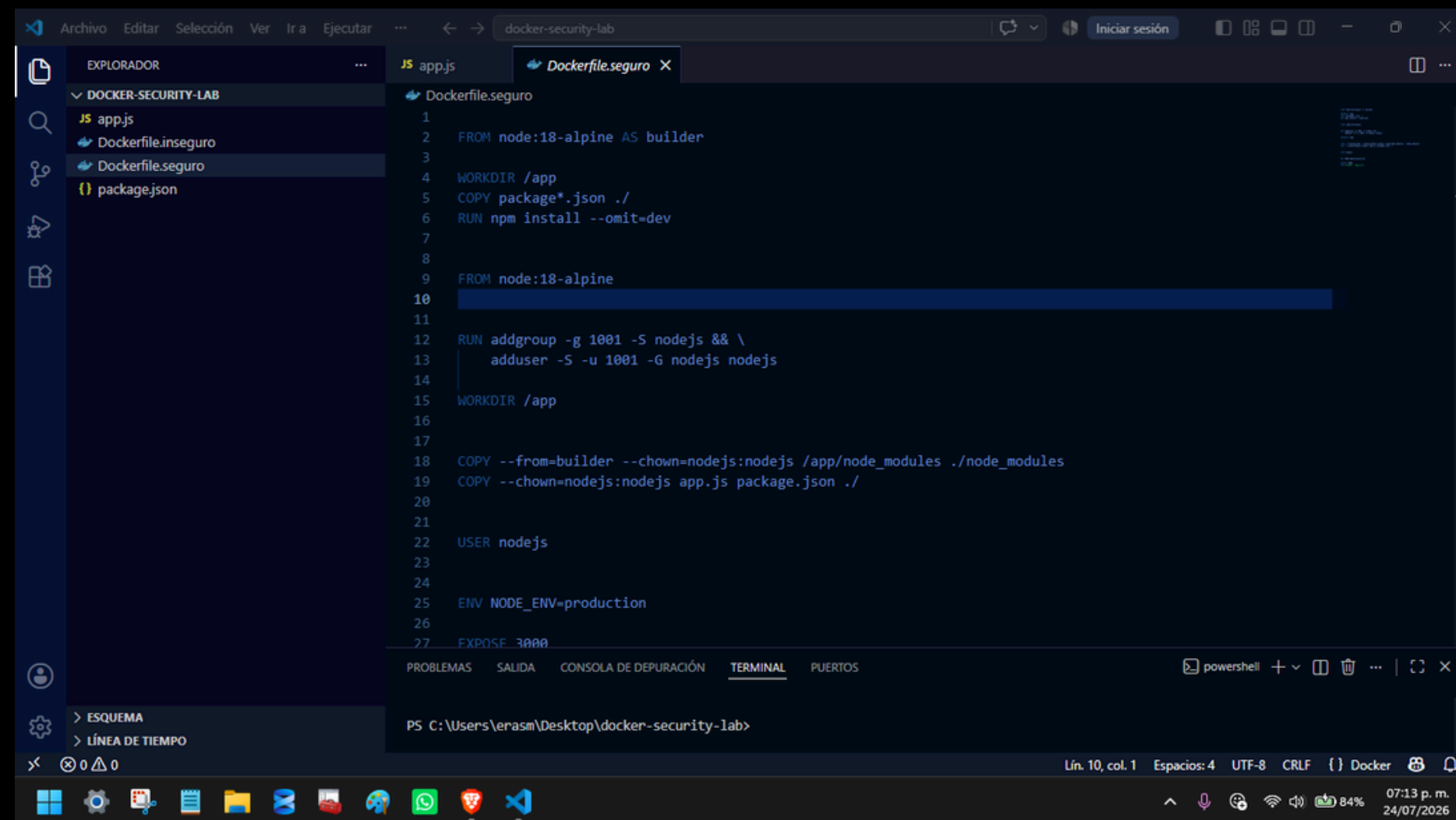


Buenas prácticas de seguridad

- Utilizar imágenes oficiales o de fuentes confiables, manteniéndolas siempre actualizadas para reducir vulnerabilidades.
- Ejecutar los contenedores con usuarios non-root, evitando el uso del usuario root siempre que sea posible.
- Aplicar el principio del mínimo privilegio, otorgando únicamente los permisos y capacidades estrictamente necesarios.
- Usar la instrucción USER en el Dockerfile para definir un usuario con permisos limitados que ejecute la aplicación.
- Evitar almacenar información sensible en variables de entorno, especialmente en entornos de producción.
- Utilizar Docker Secrets para gestionar contraseñas, claves API y demás credenciales de forma segura.
- Escanear periódicamente las imágenes con herramientas como Docker Scout, Trivy o Clair para detectar vulnerabilidades.
- Actualizar regularmente Docker, las imágenes y el sistema operativo, incorporando los últimos parches de seguridad.
- Eliminar paquetes y archivos innecesarios de las imágenes para reducir la superficie de ataque y el tamaño del contenedor.
- Limitar los recursos del contenedor (CPU, memoria y almacenamiento) para evitar abusos o ataques de denegación de servicio.

CREAR IMAGEN EJECUTÁNDOSE COMO ROOT

En Docker, por defecto los contenedores se ejecutan como usuario root dentro del sistema. Esto representa un riesgo porque, si un atacante compromete el contenedor, puede escalar privilegios y afectar al host. Para demostrarlo, se construirá una imagen que corre como root sin restricciones.



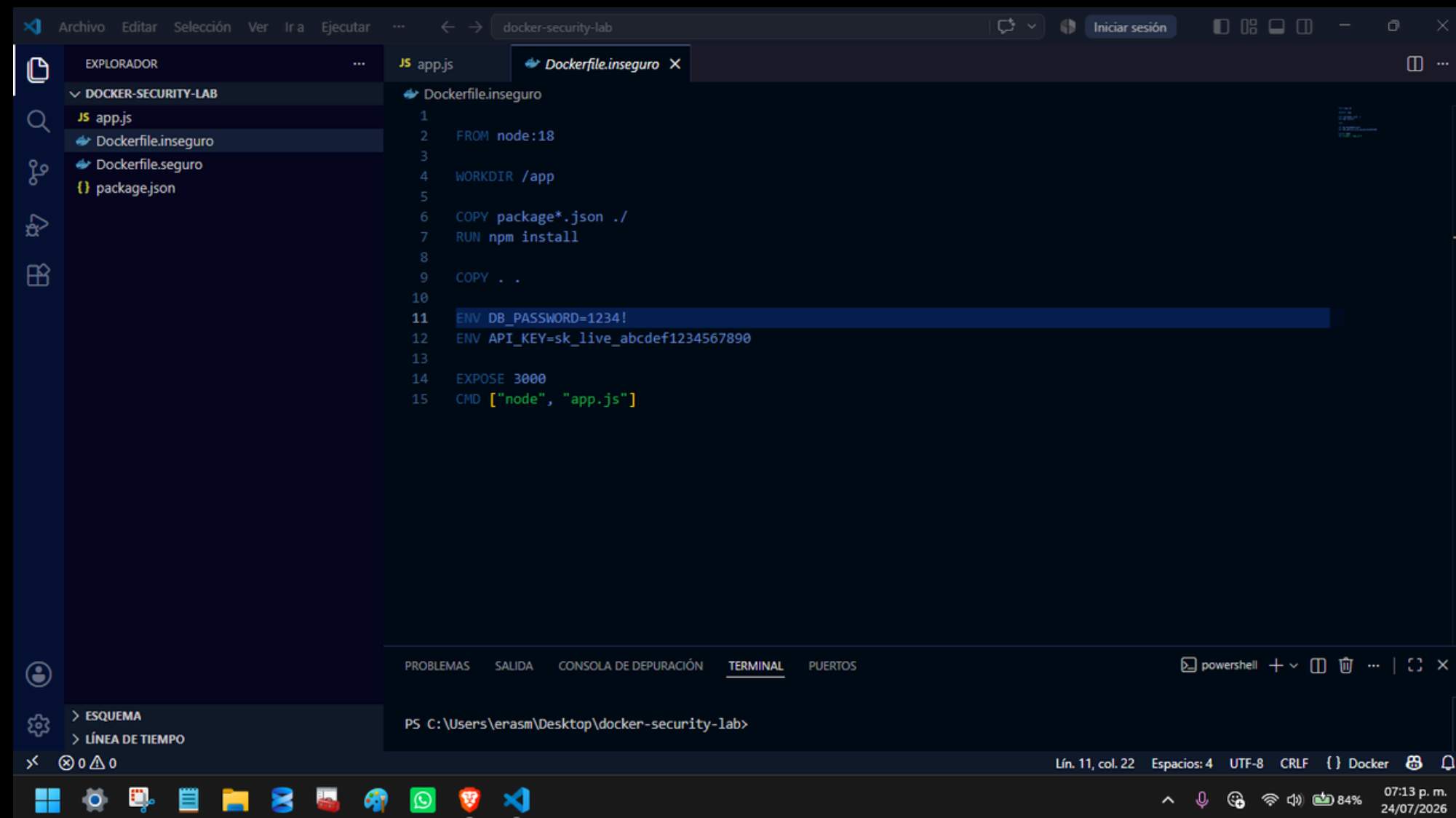
The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'DOCKER-SECURITY-LAB' with files 'app.js', 'Dockerfile.inseguro', 'Dockerfile.seguro', and 'package.json'. The code editor displays the content of 'Dockerfile.seguro', which is a Dockerfile designed to run as root. The Dockerfile includes instructions to build from 'node:18-alpine', set the working directory to '/app', copy 'package.json', install npm dependencies, add a group and user 'nodejs', and expose port 3000. The terminal window at the bottom shows the command prompt 'PS C:\Users\ernasm\Desktop\docker-security-lab>'.

```
1 FROM node:18-alpine AS builder
2
3
4 WORKDIR /app
5 COPY package*.json ./
6 RUN npm install --omit=dev
7
8
9 FROM node:18-alpine
10
11
12 RUN addgroup -g 1001 -S nodejs && \
13     adduser -S -u 1001 -G nodejs nodejs
14
15 WORKDIR /app
16
17
18 COPY --from=builder --chown=nodejs:nodejs /app/node_modules ./node_modules
19 COPY --chown=nodejs:nodejs app.js package.json ./
20
21
22 USER nodejs
23
24
25 ENV NODE_ENV=production
26
27 EXPOSE 3000
```

PS C:\Users\ernasm\Desktop\docker-security-lab>

CREAR IMAGEN EJECUTÁNDOSE COMO USUARIO NO PRIVILEGIADO

La mejor práctica es definir un usuario no privilegiado en el Dockerfile. Esto limita el impacto de un ataque, ya que el contenedor no tendrá permisos administrativos. Se crea un usuario seguro y se especifica con la instrucción **USER**

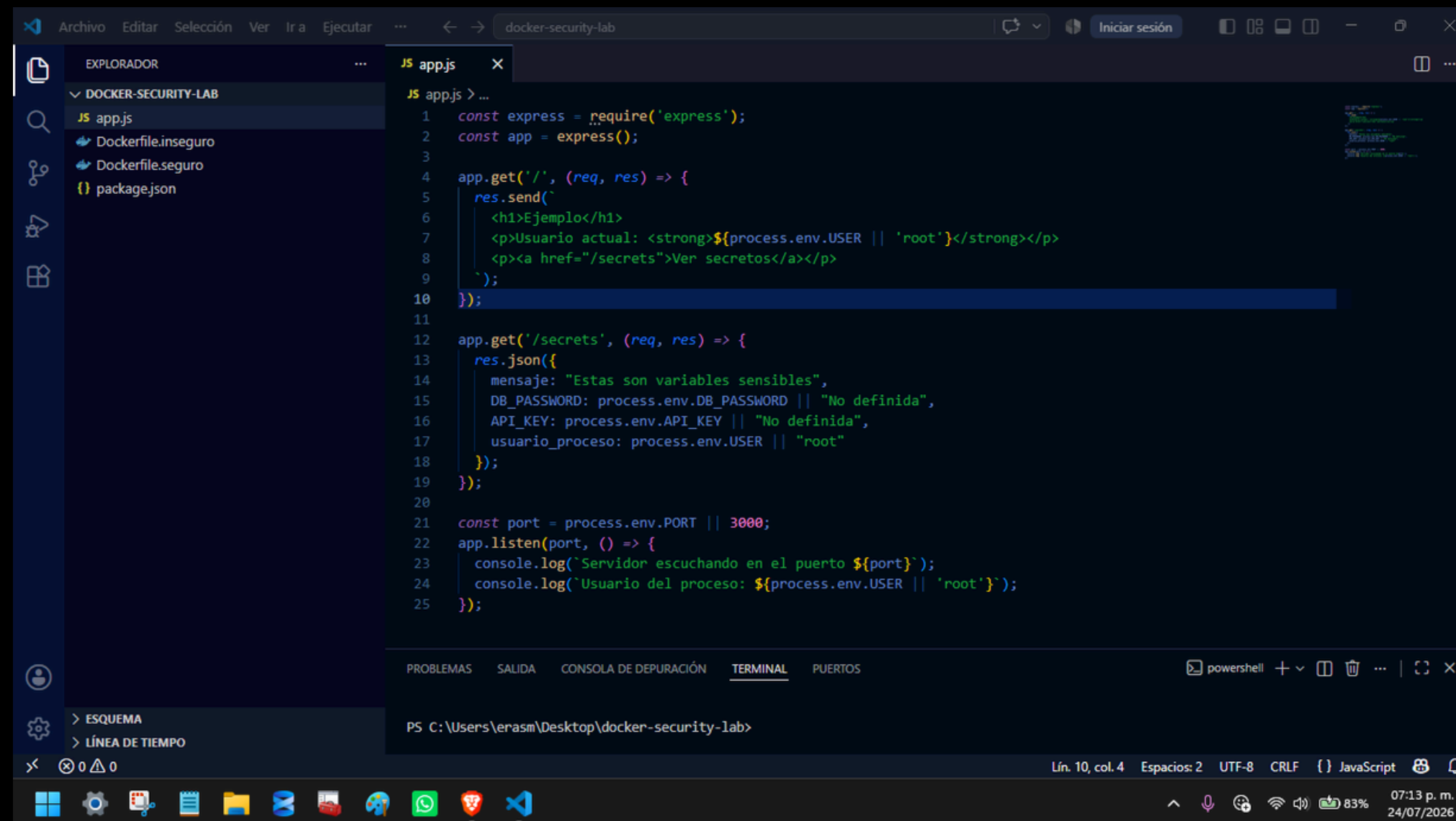


```
1 FROM node:18
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .
9
10 ENV DB_PASSWORD=1234!
11 ENV API_KEY=sk_live_abcdef1234567890
12
13 EXPOSE 3000
14
15 CMD ["node", "app.js"]
```

The screenshot shows the Visual Studio Code interface with a file explorer on the left displaying the project structure: DOCKER-SECURITY-LAB, app.js, Dockerfile.inseguro, Dockerfile.seguro, and package.json. The main editor area shows the content of Dockerfile.inseguro. The bottom status bar indicates the current line is 11, column 22, with 4 spaces, UTF-8 encoding, CRLF line endings, and the Docker extension is active. The system tray at the bottom shows the time as 07:13 p. m. on 24/07/2026.

ANÁLISIS DE VULNERABILIDADES

El análisis de vulnerabilidades permite identificar paquetes inseguros o dependencias obsoletas en las imágenes. Herramientas como Trivy o Docker Scout generan reportes que ayudan a mitigar riesgos antes de pasar a producción



```
1  const express = require('express');
2  const app = express();
3
4  app.get('/', (req, res) => {
5    res.send(`
6      <h1>Ejemplo</h1>
7      <p>Usuario actual: <strong>${process.env.USER || 'root'}</strong></p>
8      <p><a href="/secrets">Ver secretos</a></p>
9    `);
10  });
11
12  app.get('/secrets', (req, res) => {
13    res.json({
14      mensaje: "Estas son variables sensibles",
15      DB_PASSWORD: process.env.DB_PASSWORD || "No definida",
16      API_KEY: process.env.API_KEY || "No definida",
17      usuario_proceso: process.env.USER || "root"
18    });
19  });
20
21  const port = process.env.PORT || 3000;
22  app.listen(port, () => {
23    console.log(`Servidor escuchando en el puerto ${port}`);
24    console.log(`Usuario del proceso: ${process.env.USER || 'root'}`);
25  });
```

MOSTRAR REDUCCIÓN DE RIESGOS

Tras aplicar las buenas prácticas usuario no privilegiado, variables seguras, imágenes oficiales, el escaneo muestra menos vulnerabilidades críticas. Esto evidencia cómo las medidas de seguridad reducen la superficie de ataque.

```
{
  "name": "docker-security-lab",
  "version": "1.0.0",
  "description": "Laboratorio de seguridad en Docker",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

COMPARACIÓN DE AMBAS IMPLEMENTACIONES

Se comparan los resultados:

- Versión insegura: ejecuta como root, credenciales expuestas, múltiples vulnerabilidades.
- Versión segura: ejecuta como usuario limitado, credenciales protegidas, vulnerabilidades reducidas. Esta comparación demuestra la importancia de aplicar seguridad desde la construcción de la imagen.

```
Terminal
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\erasm> docker run -d --name insegura -p 3000:3000 app-insegura
07e457fa2466f4c1f4b2da24c2908fedc26284f9e4e58bbef8ee80ecf1914b92
PS C:\Users\erasm>
```

```
Terminal
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

C:\Users\erasm> docker run -d --name insegura -p 3000:3000 app-insegura
457fa2466f4c1f4b2da24c2908fedc26284f9e4e58bbef8ee80ecf1914b92
C:\Users\erasm>
```


BUENAS PRÁCTICAS APLICADAS

Finalmente, se explican las mejores prácticas implementadas:

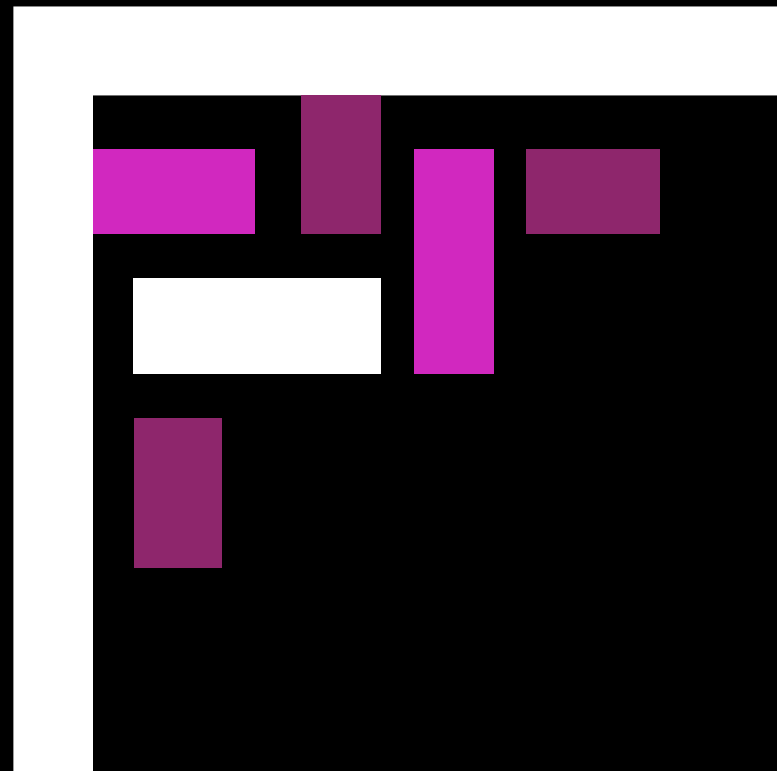
- No ejecutar como root.
- Usar imágenes oficiales y actualizadas.
- Manejar credenciales con .env o Docker Secrets.
- Escanear imágenes regularmente.
- Limitar capacidades y habilitar Docker Content Trust.

☐	☒	app-insegura	latest	4ef72e6b28f2	46 minutes ag	1.58 GB				
☐	☐	app-segura	latest	c00555e4e84a	46 minutes ag	186.32 MB				

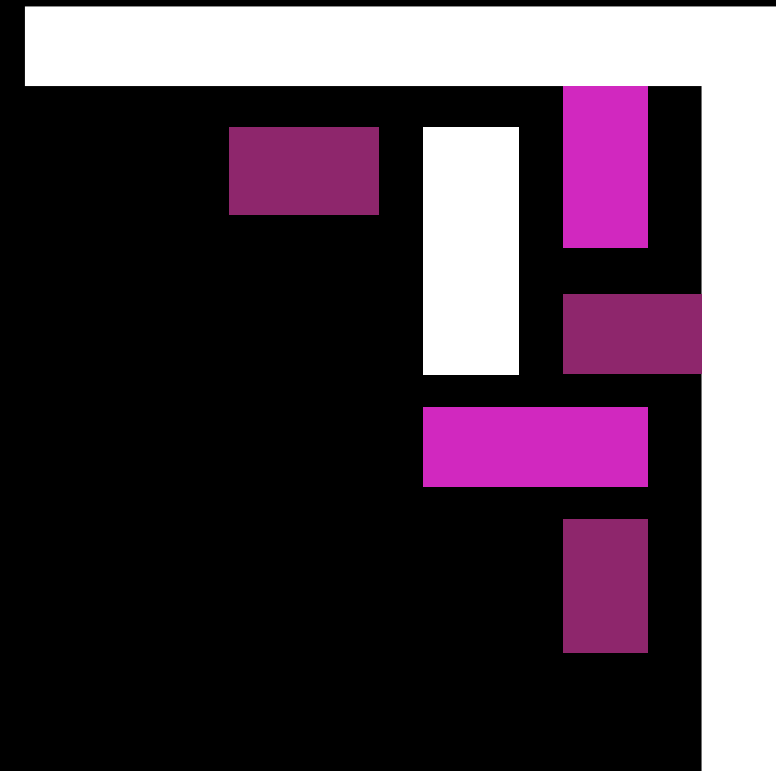
☒	☐	app-insegura	latest	4ef72e6b28f2	49 minutes ag	1.58 GB				
☐	☒	app-segura	latest	c00555e4e84a	49 minutes ag	186.32 MB				

Conclusiones

Como grupo logramos cumplir con la práctica de Docker creando y ejecutando dos versiones de nuestra aplicación: una insegura y otra segura. Pudimos levantar los contenedores en los puertos 3000 y comprobar en el navegador que ambos funcionaban correctamente. Este trabajo nos permitió entender la importancia de aplicar buenas prácticas de seguridad en entornos de contenedores y reforzó la colaboración en equipo para resolver problemas y alcanzar el objetivo final



Grupo #1



MUCHAS
GRACIAS

